DAENet: Making Strong Anonymity Scale in a Fully Decentralized Network

Tianxiang Shen*, Jianyu Jiang*, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, and Heming Cui, *Member, IEEE*

Abstract—Traditional anonymous networks (e.g., Tor) offer limited security guarantees under global passive attacks which monitor the whole network traffic to reveal who are in the same conversation. To preserve user anonymity against global passive attacks, emerging mix networks mess up network packets' orders by shuffling packets through a set of centralized mix servers, and make shuffling operations verifiable by executing shuffles in synchronized batches. However, the centralized design makes existing mix networks vulnerable to targeted DoS attacks, and the synchronization is fragile to active packet drops. We present DAENet, a decentralized mix network that resists both global passive attacks and active attacks. To handle passive attacks, DAENet leverages the confidentiality and integrity protection of Intel SGX to ensure trustworthy message shuffles at each distributed host. As we will show, naive processing of message shuffles within SGX still leaks sender identities under active attacks. To handle active attacks, we retain the use of distributed hash table (DHT) for efficient message routing, and use randomly generated shared secret to formulate different routing circuits in each communication round, preventing active attackers from tracking and revealing sender identities. We show that our system is scalable with moderate latency when running in a cluster of 10,000 participants and is robust in the case of machine failures, making it an attractive new design for decentralized anonymous communication. DAENet's code is released on http://github.com/tdsc0652/dae-net.

1 Introduction

The Internet allows convenient communications between users, but it also leads to great concerns about anonymity since communications can be censored by national security agencies and the Internet Service Providers (ISP). These adversaries can learn the identities of users who are talking to each other by recording, tampering, and analyzing communication links [1], [2], [3], [4], [5]. For example, the National Security Agency (NSA) collects the telephone number and duration of phone calls to reveal who is talking to whom [6]. Even worse, some governments intercept massive amounts of Internet traffic and block anonymous services to ease censorship [7].

Thus, it is important to build an anonymous communication system that forbids censorship. In practice, an anonymous communication system has to meet three requirements. The first requirement is **low-latency**. A high-latency anonymous network is unsuitable for use in applications involving massive interactive traffic, such as instant messaging and file sharing [8], [9]. The second requirement is **defending against passive attacks**. Passive attacks are common in communication systems where attackers observe the network traffic and packets' metadata (e.g., packet size) [10], [11], [12] to determine if two users are in the same conversation. The most powerful passive attacker, Global Passive Attacker (GPA) can observe the whole network and record all network packets and their metadata [13]. The third requirement is **defending against active attacks**. Active attacks can forge, drop or duplicate network packets, and further de-anonymize users by analyzing the changes of network status (e.g., packet emission rate) [14]. One of the most aggressive active attacks is a targeted Denial-of-Service (DoS) attack, which completely turns down core servers of anonymous communication systems.

Existing anonymous communication systems are designed to meet these requirements. The most popular anonymous network Tor [15] is a onion routing system that forward messages through a circuit of relays, constructed statically and randomly, to hide message originators. Tor satisfies the low-latency requirement as it incurs less than 2s end-to-end communication latency. Also, Tor partially meets the second requirement of defending against passive attacks, more precisely, a weak passive attacker who simply observe only a portion of the relays. Since the relays of routing circuits are distributed in the whole network, weak passive attacks cannot suspend the whole anonymous communication. Unfortunately, Tor is vulnerable to powerful global passive attacks because GPAs can learn the originator of a message by collecting and ordering all the input and output packets of relays, and analyzing all possible routing circuits to reveal a particular message circuit [16]. Last, Tor is susceptible to active packet drops which can cause head-of-line blocking behavior during packet transmission [17].

To enable stronger anonymity guarantees, the emerging shuffle-based systems (e.g., Loopix [18], Dissent [19], Karaoke [20], Riposte [21], Miranda [22] are established for providing provable protection against global passive attacks, thus meeting the second requirement. Shuffle-based systems defend against global passive attacks by utilizing centralized mix servers to collect messages from users and apply message shuffles to hide the message originators [23], [24]. To ensure correct message shuffles, these systems either use layer-based verifiable shuffles to make sure the majority of the centralized servers are trustworthy [18], [25]; or they apply cryptographically indistinguishable shuffles (e.g., using gamble circuit [26] and holomorphic encryption [27]) to avoid any information leakage in the shuffle process [21], [28].

Unfortunately, shuffle-based systems can not meet the

* *Contributes equally to this work.*

first requirement (low-latency) and the third requirement (anti active attacks) at the same time, and we owe it to two reasons. First, to reduce communication latency, some shuffle-based systems (e.g., Dissent [29]) use a set of centralized mix servers to collect messages from users and mess up the orders of input and output messages at each host through shuffling. Such design exposes an attack surface to targeted DoS attacks which can completely turn down the mix servers by repeatedly sending dummy messages, violating the third requirement. Second, some shuffle-based systems protect users from active attacks with a synchronization approach. For example, DC-Net [30] makes use of cryptographic shuffles and all-to-all broadcast to hide message patterns and enforce shuffle operations always run in synchronized batches, which is computationally expensive and has a large bandwidth cost. As a result, DC-Net supports less than hundreds of users in practice and DC-Net's end-to-end communication latency increases exponentially with growing number of users, violating the first requirement.

Recently, Trusted Execution Environment (TEE) such as Intel SGX has been applied in various security domains to guarantee the integrity and confidentiality of execution code and data [31], [32]. For example, SGX-Tor [33] leverages the confidentiality of SGX to hide the identifiers of hidden-service providers and routing circuits by storing the information of Tor relays in SGX enclaves, thus improving Tor's anonymity guarantee. Nevertheless, SGX-Tor is still vulnerable to traffic analysis attacks and requires centralized directory servers to construct circuits and conduct SGX attestations. Inspired from SGX-Tor, enclosing private information and preserving the integrity of shuffle operations within SGX enclaves may potentially facilitate us achieving the requirements in anonymous communication systems.

We present DAENet[1], the first practical anonymous messaging system based on SGX that can meet the three requirements of practical anonymous communication systems. Building DAENet is not just simply running an anonymous communication protocol in SGX. First, although SGX guarantees code integrity and data confidentiality, network-level adversaries can still observe network traffic outside enclaves and analyze the originators of messages. Second, active attackers outside the enclave can still drop or delay network packets. Even worse, active attackers might completely abort from its services and withdraw from the network to hurt the liveness of the system.

To defend against global passive attacks, DAENet proposes a *Stealthy Peer-to-Peer (P2P) Network* with two design points. First, the stealthy P2P network hides the identities of participants when no active attacks occur. A participant joins the network and connects to a series of members whose locations are unknown to the participant. The identity of the participant and its neighbors' information are enclosed inside SGX, such that malicious attackers and even the participant's malicious host cannot learn the identity of that participant in the network. Second, to hide message patterns, our stealthy P2P network enforces trustworthy message shuffles that mess up the orders of input network packets at each distributed host, and obliviously disseminates output packets to the neighbors of each participant. With the above-mentioned designs, we prove that our stealthy P2P network produces oblivious packet transmission under global passive attacks for all participants (§5.1).

In addition to global passive attacks, we identify two threats of active attacks targeting at DAENet's anonymous communication. The first threat is *Eclipse Attacks* that arbitrarily drop network packets to partition the network. As DAENet's stealthy P2P network relies on control messages to check the liveness of neighbors, manage routing tables and further maintain a consistent view of membership, the network will get partitioned if some control messages are maliciously blocked or dropped. Confronted with such threat, DAENet adopts an efficient detection-based design to detect malicious drops of packets. Each participant periodically sends a self-loop message to a random neighbor and routes back. After that, the participant consults each on-path relay to send back a receipt, proving that it had honestly forwarded the loop message rather than maliciously dropped the packet.

The second threat is *Traceback Attacks* where active attackers collaborate with participants to reveal a targeted sender's identity. In such attack, a compromised participant holds a long-lived connection with a message sender in the network, while at the same time active attackers delay or drop packets to analyze the static routing circuit between them, based on the observation of whether the compromised participant has received the message from the sender in time or not. To tackle this vulnerability, we use a *dead drop* abstraction that randomly chooses one node in the network to exchange instant messages in each communication round, such that adversaries cannot learn the exact messaging circuit and further reveal a sender's identity.

We implemented DAENet with 5.2k LoC in C++ on Linux. DAENet admits participants with a membership protocol that attests the SGX code integrity and assists in routing table creation. DAENet uses a secure dialing protocol to initialize conversations without leaking private information to adversaries. DAENet also tolerates network churn and machine failure to guarantee the liveness of its anonymous service. We make the following contributions in this paper:

- We leverage commonly available trusted execution capabilities of SGX enclaves to build the first low-latency anonymous communication system that defends against both passive and active attacks.
- We conducted large scale evaluation and proves that DAENet is efficient to scale to a large number of participants. DAENet incurs 2.2s latency with 10,000 participants. Compared with Loopix [18], a state-of-art shuffle based system, DAENet incurs 6X $\sim$ 7X lower latency than Loopix.

The remaining of this paper is structured as follows. §2 describes the background of anonymous communication systems, the underlying P2P network of DAENet and Intel SGX. §3 introduces the roles of users and security primitives of DAENet. §4 describes detailed DAENet's anonymous protocols. §5 gives an end-to-end security analysis. §6 is the performance evaluation of DAENet. §7 discusses potential limitations of DAENet. §8 introduces the related work and §9 concludes our work.

1. DAENet is for a **D**ecentralized, **A**nonymous and **E**fficient network.

Category		End-to-End Latency (s)	Anti GPA	Anti-Reveal Circuit ID	Anti-Packet drop (liveness) / (Anonymity)	Anti Targeted DoS
Relay-based	Tor [15]	0.25 ∼ 1.5	×	×	√ / ×	×
	SGX-Tor [33]	0.525 ∼ 3.15	×	√	√ / ×	×
	ShadowWalker [34]	> 4	×	×	√ / ×	√
	AP3 [35]	——	×	×	√ / ×	√
Shuffle-based	Loopix [18]	6.8	√	√	√ / ×	×
	Riposte [21]	> 3600	√	√	× / √	×
	Dissent [29]	1.3	√	√	× / ×	×
	Atom [36]	30.0	√	√	√ / √	×
	Karaoke [20]	6.0	√	√	× / √	×
	DAENet	2.2	√	√	√ / √	√

Table 1: Comparison of DAENet to relay-based and shuffle-based systems. "√" indicates that the system can handle such vulnerability, while " × " is on the opposite.

2 Background

2.1 Anonymous Communication Systems

Existing anonymous communication systems can be classified into two categories: relay-based systems and shuffle-based systems. We compare these systems with regard to the three requirements of practical anonnymous communication in Table 1.

Among the relay-based systems, Tor [15] is the most popular anonymous communication system ever deployed, with an estimated eight million daily active users [37]. Tor incurs less than 2s communication latency, satisfying the first requirement. However, these is a large body of work that examines global passive attacks against Tor, targeting at de-anonymize sender identities [10], [11], [38], thus violating the second requirement. Tor is also susceptable to active attacks that arbirarily drop network packets, casuing head-of-line blocking during data transmission. A recent work also shows Tor's susceptibility to targted DoS with bandwidth amplification [39]. As an improved work of Tor, SGX-Tor [33] has sligtly higher latency (around 3s), hides sensitive information of Tor components (e.g., circuit ID) but also inherits Tor's susceptibility and violates the second and third requirement as well. Different from Tor-related designs, other relay-based anonymous communication systems, such as ShadowWalker [34] and AP3 [35] are built upon a structured P2P network to eliminate the concern of targeted DoS attacks. However, these systems cannot meet the second requirement because global passive attackers can still observe the whole network traffic and correlate users in the same conversation.

In contrast to relay-based systems, shuffle-based systems leverage message shuffles to defend against global passive attacks, thus meeting our second requirement. Riposte [21] is a stronge anonymity system that uses Private Information Retrieval (PIR) techniques to defend againt active attackers [40], [41], meeting the third requirement. However, Riposte assumes that the participants can tolerate its hours of lantency to achieve strong anonymity, violating the low-latency requirement. Similar to Riposte, Atom enforces stronge anonymity guarantees but incurs high communication latency (30s). Loopix [18] uses cover traffic and poison-mixing strategy to efficiently shuffle real messages with dummy message, defending against global passive attacks. However, Loopix uses designated message boxes to collect client's messages and disseminate messages to its centralized mix servers, suffering from targeted DoS attacks. Dissent [29] is the most efficient shuffle-based anonymous network as it incurs less than 2s communication latency which is comparable to Tor. As Dissent uses centralized nodes to collect and shuffle messages and then broadcast messages to the network, it is vulnerable to targeted DoS attaks, violating the third requirement. Karaoke [20] is a recent work that leverages differential privacy techniques to add noise messages and shuffle these noise messages with real messages through layers of mix servers. Although it uses verifiable shuffles to protect shuffle operations from active attacks, it also suffers from targeted DoS attack to its set of fixed mix servers that executes message shuffles.

2.2 Structured Peer-to-Peer Network

Recent peer-to-peer anonymous communication systems (e.g., ShadowWalker) utilize a distributed hash table (DHT) such as Chord [42] and Pastry [43] to route messages in a structured peer-to-peer network. In a DHT, nodes are assigned identifiers and a range of values they are responsible for. Nodes only have knowledge about a fraction of the network called *neighbors* which are stored in routing tables. When a node tries to lookup a value, it first checks its routing table and asks a neighbor who is numerically closest to the value. The neighbor, in turn, repeats this process. The lookup ends until the receiver that owns the value is found.

DAENet uses Chord for efficient lookup in a structured peer-to-peer network. In Chord, each participant joins the network by sending a *join* request to a known Chord node. The Chord node will assign an identifier to the participant and help the participant set up its routing table. The identifier space is within $[0, 2^b)$, and b is chosen according to the scale of network. Each participant knows only a fraction of other participants in the network. Specifically, for a participant with identifier idx, it is connected to b neighbor nodes who have the numerically closest identifier to $idx + 2^i (0 \leq i < b)$. Note that not all slots in the identifier space have to be used: Each slot in the identifier space, named as S, is mapped to a participant node who has numerically closest identifer to S (i.e., *Map(S)* = $ClosestNode_{idx}$). In DAENet, we call the neighbor nodes of a participant *successors* and the participant itself is called the *predecessor* of all its neighbor nodes. To maintain a con-

sistent view of membership, participants periodically send query messages to check the liveness of their successors and remove inactive successors from their routing tables. Unless specifically pointed out, we denote N as the total number of participants in the network.

Although Chord facilitates efficient lookup, the protocol itself does not provide anonymity. By analyzing the incoming and outgoing network packets, global passive attackers can learn the successors of each node and further reveal the entire structure of the network. With an explicit network structure, adversaries can easily drop targeted users' network packets to block the anonymous service.

2.3 Intel SGX

Intel Software Guard eXtension (SGX) [31], [32] is a popular security hardware available on commodity CPUs. It provides secure execution by putting data and code inside a container called *enclave*. The *enclave* is isolated from privileged software such as the operating system (OS), firmware and hypervisor, so that the protected code and data cannot be easily tampered with or revealed from outside. The untrusted part and the enclave run as isolated processes, communicating through a narrow and well-defined interface. A process running outside the enclave can invoke an SGX *ECall* to switch its execution into the enclave; a process running in an enclave can invoke an *OCall* to switch its execution outside the enclave. Besides, SGX also provides remote attestation [44] to verify that a particular piece of code is running in a genuine SGX-enabled host.

3 Overview

We begin with describing the threat model and the roles of participants in DAENet. Then we discuss DAENet's security goals and corresponding defense techniques.

3.1 Threat Model

We consider sophisticated and well-resourced adversaries in the network, who attempt to determine if two participants are communicating, given that the message sender or receiver may collude with the adversaries. Therefore, we consider adversaries with two distinct capabilities: global observation and traffic control. DAENet requires an adequate number of honest participants to ensure complete message deliveries. Specifically, DAENet requires at least $\mathcal{O}(\log \mathcal{N})$ out of $\mathcal{N}$ honest participants, in which the coefficient depends on the *rounds* of communications and the total number of participants. Similar to other SGX-enabled systems [33], [45], SGX firmware and the code running in SGX are trusted, SGX-related side-channel attacks (e.g., cache and timing attacks) are out of the scope of this paper.

3.2 Participants As Protocol Parties

In DAENet, all participants equally act as part of the protocol with no designated powerful managers, such as administrative servers or message boxes in previous anonymous communication systems [18], [29]. A participant can play multiple roles at a given time, determined by whether it is communicating or not. Therefore, targeted DoS attackers

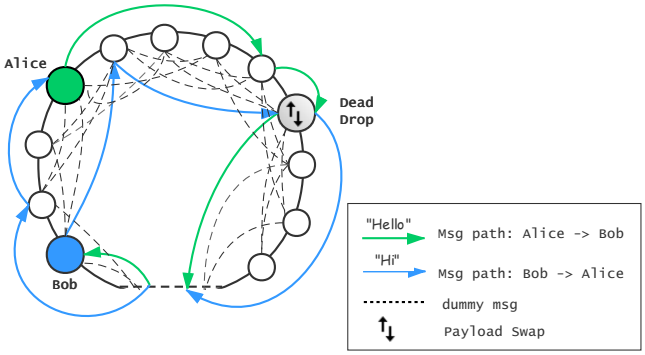


Figure 1: An example of DAENet dead drop messaging. In round R, Alice and Bob separately sends two close-loop messages while exchanging their message payload at a designated dead drop node.

cannot distinguish and attack a specific participant who is playing a critical role of the network. There are totally three roles in DAENet: *Relay*, *Session Node* (i.e., sender/receiver) and *Dead Drop Node*.

Relays and Session Nodes Relays are idle participants. They are not in any conversations and only responsible for forwarding messages in the network, including both application messages (e.g., instant messages) and underlying P2P control messages (i.e., messages for maintaining DAENet's structural topology). In contrast, session nodes are participants in conversations. A participant acts as either a relay or a session node in DAENet.

Dead Drop Nodes help exchange message content between pairs of session nodes. To initialize a set of dead drop nodes, two DAENet participants first negotiate a randomly generated shared secret. The shared secret is used for generating a sequence of $DeadDrop_keys$. Since DAENet enables deterministic *KEY-ID* mapping by building on top of Chord (§2.2), $DeadDrop_keys$ are deterministically mapped to a series of nodes. Hence two session nodes can eventually agree on the same sequence of dead drop nodes in the network. All participants can act as dead drop nodes in DAENet. The duty of a dead drop node is ephemeral and will become invalid as soon as the dead drop node completes message exchanging.

Figure 1 shows a flow of dead drop messaging in a communication round. With reference to the $DeadDrop_keys$, Alice and Bob send their messages to the same designated participant (i.e., the dead drop node) of the network. The dead drop node waits for two messages coming and then exchanging the message content and sending them back to originators.

3.3 Security Goals and Privacy Approaches

The main purpose of running an anonymous communication system in a fully decentralized network is to remove the targeted DoS concern of centralized servers. Moreover, to enable stronger anonymity guarantees, we additionally handle the following three critical vulnerabilities.

3.3.1 *Protecting Against Global Passive Attacks*

Global Passive Attackers (GPAs) are proven to be the strongest passive adversaries in the network who keep

eavesdropping on network traffic among all the participants, trying to find a circuit of a particular communication and link corresponding participants. To determine if two participants are in a communication, GPAs may conduct prefix hijacking [46] to intercept network traffic and then use off-path statistical analysis [47] to sort messages. For example, in a typical passive traffic analysis attack, GPAs inspect every message of the network and keep observing the load of each participant. Since the overall traffic patterns are not particularly distorted at each hop, GPAs can correlate the incoming and outgoing time of every input and output message. Given sufficient time, GPAs can restore all message circuits for all communication sessions. Besides, GPAs can also learn the emitting rate of messages at each host. A high emitting rate might reveal a potential message sender when other parts of the network is idle. Built on top of a structured P2P network, DAENet's topology is explicit to GPAs. Adversaries can use the above-mentioned approaches to deduce all the neighbors (i.e., successors) of each node through statistical analysis.

Defend Against GPAs. DAENet prevents GPAs from correlating any pairs of senders and receivers in conversations, even with an observed network topology. The shuffling process works as follows: Alice maintains shuffle pools for each successor node in its routing table. Upon receiving a message m, if the next hop of m is the ith successor of Alice, then the message is pushed to the ith shuffle pool belonging to Alice's ith successor. In each protocol run, Alice pulls messages from each successor's shuffle pool and sends them out with a probability p. Given a threshold α, if p is smaller than α, Alice will not pull a message from the ith successor's shuffle pool, and Alice will directly send a dummy message to its ith successor. Note that Alice may hold none messages in its ith shuffle pool at a given time, if so, Alice will send a dummy message to its ith successor.

3.3.2 Detecting Active Eclipse Attacks

Eclipse attack is a major vulnerability in a P2P network. The adversary arbitrarily drops messages among participants to partition a fraction of them from the network, or assigning malicious nodes to harass the routing table of a specific participant. Since DAENet is built on top of a structured network which relies on nonstop sending of control messages to refresh the membership and maintain its topology, if some control messages are maliciously dropped, the membership will get partitioned and communications will fail due to incomplete message deliveries. In addition to an application-level failure, in a partitioned network, the anonymity set is smaller and the adversary will have a larger chance to de-anonymize targeted participants.

Defend Against Eclipse Attacks. DAENet resists eclipse attacks by using *loop* messages to actively check the membership of each participant. A *loop* message contains a randomly generated secret s which is only known to the participant itself. Each participant periodically sends a *loop* message to a random successor node, and the successor node repeats this process by forwarding the message to one of its successor node. The process ends until the *loop* message finally reaches the original sender. If the *loop* message successfully routes back, all the on-path relays are considered to be secure; Otherwise, the participant consults each on-path relay to ask for a *proof-of-forwarding* which must contain s. If a relay fails to provide such proof, the relay is considered to be potentially malicious, and all the predecessors of that relay will be notified to remove the relay from their routing tables (§4.3).

3.3.3 Preventing Active Traceback Attacks

Similar to other DHT based lookup schemes, the routing circuit between any two participants is deterministic in DAENet. This poses a threat to sender anonymity: an active attacker might collaborate with a receiver to reveal the identifier of a sender by reconstructing the routing path between them. In the *traceback attack*, a compromised message receiver holds a long-lived connection with another participant in the network and keeps sending instant messages. At the same time, the network adversaries delay or drop messages between the sender and receiver to reveal the routing circuit, based on the observation of whether the compromised receiver has received the message from the sender in time or not (§4.4).

Specifically, in this *traceback attack*, a malicious receiver R collaborates with active attackers to reveal the identifier of sender S. Denote a message path $||C_i|| \Leftarrow < S, P^1, P^2, ..., P^{i-1}, P^i, R >$ as the routing circuit that links the malicious receiver R and the victim sender S. Since the network topology is explicit to adversaries with a global view, R can periodically, yet slowly drops messages from its predecessors. If R drops an instant message from one of its predecessors and receives no messages from S in next communication round, then R learns that this predecessor is actually P^i - the participant that acts as the previous hop of R in $||C_i||$. Now that the path $< P^i, R >$ is revealed, the adversaries try to find P^{i-1} by dropping or delaying messages from P^i's predecessors. By repeating this process, the malicious receiver R will ultimately reveal the sender S. The traceback attack succeeds when R can receive the messages even all messages from S's predecessors are blocked.

Defend Against Traceback Attacks. To defend against traceback attacks, DAENet enforces participants to exchange message payload at random locations in the network with the help of *dead drop nodes*. Communication happens in *rounds*. In each communication round, two session nodes send messages to a dead drop node, the dead drop node swaps the payload of these two messages and sends them back to corresponding message originators.

4 Design

4.1 Membership Protocol

Participants join the DAENet network as normal relays at the beginning. When a node wants to join DAENet and uses the anonymous service, it first finds a member node through an out-of-band peer discovery service (e.g., a public forum). We call that member node the *guarder node*. A *guarder node* serves as an attestation server to verify whether an unmodified DAENet's program is executed inside a real SGX host. If the node passes the attestation, the *guarder node* replies with an automatically generated identifier, which indicates the node's location in the DAENet network.

Specifically, node *i* joins DAENet with three steps. First, *i* creates its DAENet enclave, generates its symmetric key sk_i in the enclave and seals sk_i to local storage. Second, *i* sends a *join* request to the *guarder node*. The *guarder node* does a standard SGX remote attestation and succeeds with a signed report from the Intel IAS. Third, the *guarder node* verifies the report, generates an identifier of node *i* and encrypts it with sk_i, and sends both the sealed identifier and attestation report to node *i*. If node *i* passes the attestation, it will send a lookup request with its symmetric key sk_i to the *guarder node* to construct its routing table. The *guarder node* helps node i constructs its routing table by running a standard Chord member join protocol, and notifies a fraction of nodes that precede *i* that a new participant joins the network. Note that sk_i is distributed to all the predecessors of node *i*, which is used for encrypting messages that are sent to node *i*.

Utilizing the above approach to admit regulated participants may have a potential risk: the channel to join the network is public to adversaries, thus a node may discover a fake DAENet participant and join a fake DAENet. Also, if a malicious participant is chosen as a *guarder node* and serves as an attestation server to admit new nodes, it might refuse to admit benign nodes or try to admit specific participants (most likely be malicious). DAENet uses mutual attestation to detect malicious *guarder nodes*. A newly joined node will also serve as an attestation server to verify the integrity of its *guarder node*. The mutual attestation is triggered when a *guarder node* sends the attestation report to node *i*, at the same time it provisions a self-attestation request to node *i*. Now node *i* acts as an attestation server, sends the report of the *guarder node* to Intel IAS and waits for a signed report. Note that the attestation to a *guarder node* is hardcoded into the membership protocol and the execution is enforced unless the *guarder node* withdraws from the network. Since SGX remote attestation can help verify the integrity of the running SGX code, if a malicious *guarder node* refuses to admit benign nodes or tries to admit specific participants, the malicious *guarder node*'s code integrity is broken. Hence the malicious *guarder node* will fail to pass the attestation. The failure of passing the SGX attestation helps the new participant take action quickly:

(1) Alert users in the out-of-band peer discovery service to reduce the confidence of that malicious *guarder node*, or immediately end up contacting with that *guarder node*.

(2) Retry the admission process by switching to a new *guarder node* (hopefully, one that is not malicious).

This policy limits the influence a malicious *guarder node* can do during admission, allowing DAENet to admit trustworthy participants running correct protocol. Note that DAENet can only admit SGX-enabled hosts as participants and will reject hosts without SGX.

We notice that an SGX may be compromised because of SGX vulnerabilities [48], further compromising the anonymity provided by DAENet. DAENet solves this problem using two approaches. First, such vulnerabilities can usually be fixed through CPU microcode updates [49], and such updates increase the Security Version Number (SVN) used for attestations. DAENet's guarder node checks the latest SVN within the network and rejects nodes with SVN less than this value during attestations, so nodes with out-of-date microcode (i.e., contain potentially compromised SGX) cannot join the network. Second, for vulnerabilities that cannot be fixed through CPU microcode updates, Intel returns a revocation certificate list during attestations. DAENet rejects attestation reports signed by these certificates and avoids the admissions of nodes with SGX vulnerabilities that cannot be fixed.

4.2 Shuffling: Prevent Circuit Correlation

To prevent global passive attackers from linking two session nodes, DAENet's shuffling strategy is designed so that, for any message that traverses a participant, adversaries cannot identify its preceding or succeeding messages and further reconstruct the entire routing circuit of a conversation. We define *Sender-Receiver Unlinkability* as the inability for global passive attackers to distinguish whether $\{S_{real} \rightarrow R_{real}\}$ or $\{S_{real} \rightarrow R_{other}, S_{other} \rightarrow R_{real}\}$ for a real message sender S_{real}, a real message receiver R_{real}, and other participants S_{other}, R_{other}.

DAENet preserves sender-receiver unlinkability with a trustworthy *shuffling* protocol. The core idea to is to mess up the message orders and hide communication patterns with dummy messages. The shuffling protocol requires each participant to maintain *shuffle pools* for each of its successors. For each input message, Alice first decrypts the message by using its own symmetric key, recalling that a sender will encrypt its messages with the symmetric key of the next hop (i.e., successor). Then Alice searches for the next hop of the message with reference to the identifier of the receiver node. If the next hop for that message is the *i*th successor of Alice, then the message is pushed to the *i*th shuffle pool belonging to Alice's *i*th successor.

In each protocol run, Alice totally pulls $\log \mathcal{N}$ messages from each of its shuffle pool by α, which is the *expected shuffle rate* - a parameter that indicates the probability of choosing a message from a shuffle pool. In other words, the *expected shuffle rate* implies whether Alice will send an message to its *i*th successor or not. If Alice does not pull a message from the shuffle pool of the *i*th successor, then Alice encapsulates a dummy message and sends the dummy message to its *i*th successor. Note that it's likely for Alice to hold none messages in its *i*th shuffle pool. In that case, Alice will directly send a dummy message to its *i*th successor.

More precisely, when Alice receives a message x, it

(1) decrypts x by using its own symmetric key key_A,

(2) discards x if x is a dummy message. Otherwise, runs the Chord lookup protocol to search for the next hop of message x. Let x_id be the identifier of the next hop, Alice resets x's message header to x_id, and pushes message x to x_id's shuffle pool.

(3) randomly pulls l messages from shuffle pools of each successor with equal probability α.

(4) encapsulates dummy message dmy_i if Alice does not pick a message from shuffle pool p_i.

(5) encrypts l messages with the symmetric key of corresponding successors and sends them out.

Denote $\mathcal{N}$ as the total number of participants in the network and k as the number of empty shuffle pools of Alice's all successors. Derived from previous statements, Alice pulls messages from $\log \mathcal{N} - k$ shuffle pools in each protocol run. The pulling process takes the form of Binomial

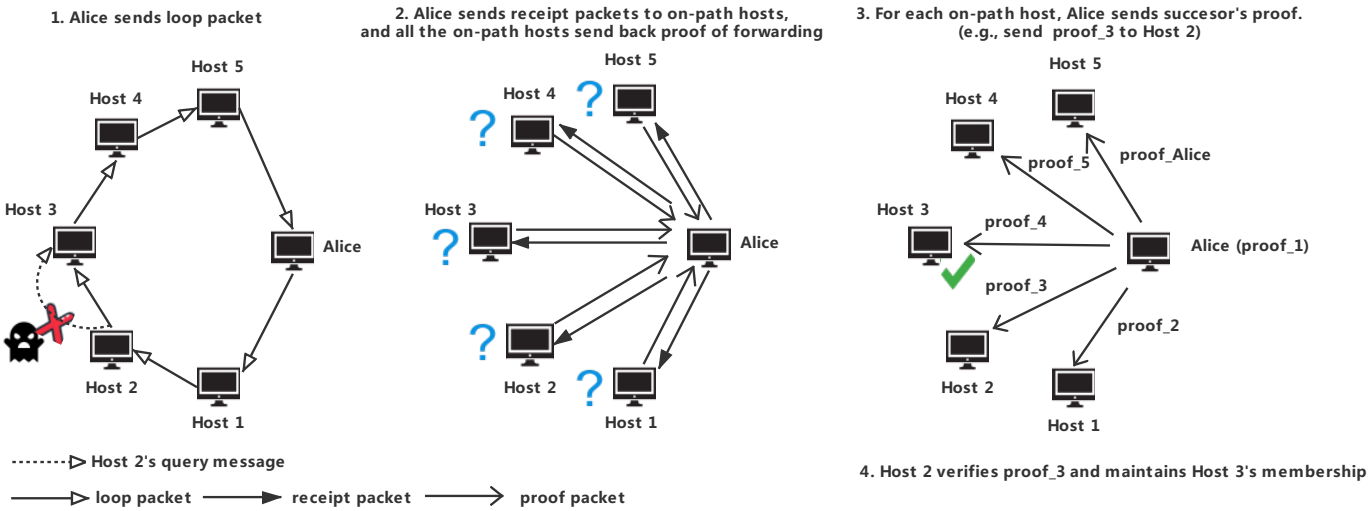


Figure 2: Each participant periodically, and randomly sends loop messages to detect malicious message drop. Here, even if Host 2's query message to Host 3 is maliciously dropped, after verifying Host 3's *proof of forwarding*, Host 2 determines that Host 3 is actually online, hence maintains Host 3's membership.

distribution $\mathcal{X} \sim B(\log \mathcal{N}, \alpha)$ where the discrete probability α is the *expected shuffle rate*. In each round, the expected total number of real application messages and dummy messages for Alice to send is $\alpha(\log \mathcal{N} - k)$ and $k + (1-\alpha)(\log \mathcal{N} - k)$, respectively.

Consider the case where global passive attackers keep observing network traffic and are capable to learn the exact number of messages in Alice's host. We define a scenario O_{x,x_1} as an adversary observing Alice's host in which message x arrives and mixes within Alice's shuffle pools. The adversary then observes $\log \mathcal{N}$ messages sending out and tries to correlate x with one of the outgoing message x_1, which is from the same conversation. Supposing the adversaries have a high confidence of message x being a real message (rather than a dummy message), the following claim gives an probability on which the adversaries correctly link the previously observed message x with one of the outgoing message x_1.

Claim 1. *Let y be the number of messages in a host in scenario O_{x,x_1}. Denote the number of non empty shuffle pools in a node as t, and let k be the number of empty shuffle pools. After shuffling, the probability of correctly linking x to one of the outgoing message x_1 is*

$$Pr\,(x = x_1) = \frac{\alpha[\sum_{c=1}^{y-t-1} \frac{1}{c} Pr(C_x{=}c)]}{t+k} \tag{1}$$

in which

$$Pr\,(C_x = c) = \binom{\text{y-t}}{c} (\frac{1}{t})^c (\frac{t-1}{t})^{y-t-c} \tag{2}$$

Note that $t+k$ is the total number of outgoing messages from Alice's host. All of the outgoing messages have equal opportunity of being the previously arrived message x, independent of the arrival time of x. This ensures that the arrival and departure time of the messages cannot be linked, so that adversaries learn no sensitive information by conducting traffic analysis. Note that the probability $\frac{1}{y}$ is the upper bound for an adversary to correctly link the input message x and the corresponding output message x_1. Thus, continuous observation of Alice's traffic leaks no sensitive information other than the present number of messages in Alice's host.

We use the above claim and a security metric *likelihood* to give an end-to-end anonymity evaluation of defending global passive attacks in security analysis (§5). To conclude, by randomly picking real messages from shuffle pools and disguising unpicked real messages with dummy messages, we obfuscate the adversary's view and decrease the probability of successfully correlating the input and output messages.

4.3 Handling Missing Shuffle Messages

Running the shuffling protocol within SGX guarantees the integrity of shuffling operations. However, active attackers might attempt to subvert the protocol by refusing to send messages, resulting in some loss of shuffle messages. When some shuffle messages are lost, adversaries can conduct eclipse attacks to ruin the network construction by preventing a target participant from obtaining a consistent view of neighbor nodes. Although adversaries cannot drop specific messages because we encapsulate messages to the same format within SGX, adversaries might randomly drop messages to ruin the process of network construction, harass the membership and ultimately deny the service of DAENet.

There are totally two cases: first, if an adversary drops a dummy message or an application message, the membership will not be affected. This is because the underlying P2P topology relies only on control messages (specifically the query messages §2.2) to refresh the membership, dummy messages and application messages are not involved in the membership maintenance process. Second, if some control messages are maliciously dropped by active attackers, the membership will get partitioned if DAENet retains Chord's design by using one query message to determine whether a participant's neighbors are alive or not because a participant

will remove the "offline" neighbors from its routing table even if the neighbors are actually online.

To prevent such vulnerability, DAENet's participants periodically send *loop* messages to detect potential eclipse attacks. For a participant S in the network, S specifies a random bit string b_s and constructs a routing path by selecting the $b_s th$ neighbor deterministically as the next destination in each hop, until encountering the sender S itself. Participants at each hop record both b_s and a round number as a receipt of forwarding the loop message. The algorithm of sending a loop message is shown in algorithm 1.

If the loop message completes its loop and routes back to S, this indicates that all the participants on the routing path are alive, then the sender will send a *proof-of-forwarding* to every on-path participants to notify that their $b_s th$ successors is valid. On the contrary, if the loop message fails to complete the loop back, this reveals that at least one of the participants on the path misbehaved. Then S sends *receipt* messages to all the on-path participants to ask for their proofs of forwarding the *loop* message. Since the on-path relays are deterministically selected by choosing the $b_s th$ successor at each hop (see algorithm 1), S can find all the on-path relays by iteratively asking for the $b_s th$ successor at each hop. Note that the random bit string b_s contained in the loop message is originated within SGX, even a malicious host cannot see the plaintext of that bit. Hence the random bit string in loop message is unforgeable. Upon receiving the *receipt* message from S, each on-path participant sends back proof of forwarding independently. The sender verifies the bits in each proofs and reports the verification results back to the participants.

In Figure 2, Host 2's query message to Host 3 is dropped by an adversary. Instead of removing Host 3 from its routing table immediately, Host 2 waits for a *proof-of-forwarding* within a fixed time interval. When Host 2 receives such proof from Alice that Host 3 forwarded its loop message successfully, Host 2 learns that its query message may have been dropped due to either unstable network environment or malicious active attackers, and will keep Host 3's membership maintained. If Host 3's proof is invalid, then Host 2 can safely remove Host 3 from its routing table and re-fetch a new successor. All participants that proceeds Host 3 will soon be notified within several protocol runs by running Chord's topology maintenance protocol.

There are two cases for malicious message drops. The first case is when active attackers entirely block the traffic from a participant to all its successors. In that case, the loop message is dropped and fails to complete its loop back, and the participant will successfully detect such misbehaves. The second case is when active attackers selectively drop messages sent by a participant. To handle that case, we enforce probabilistic defense by sending loop messages according to an *expected detection probability* discussed below.

When to send loop messages? The sending of loop messages happens in rounds, in each round, a participant sends several loop messages alongside other messages, including the real application messages and network control messages. The number of loop messages that a participant should send in round rnd is determined by τ, which is the *expected detection probability* chosen by the participant. Specifically, for each message that is ready to sent out, that message has a probability τ of being a loop message.

Denote a participant S sends m messages in round rnd, there are totally $\lceil \tau \cdot m \rceil$ loop messages contained in the m messages. Consider the case where malicious active attackers selectively drop messages with probability β, hence $\lceil \beta \cdot m \rceil$ messages are dropped in round rnd. In that case, if one out of all loop messages is dropped, the participant S can still detect it with probability P_r.

Algorithm 1: SendLoop(msg)

```
S_id ← the identifier of sender S
b_s ← a random bit string generated by S
rnd ← the round of msg chosen by S
record(t) ← record t as a proof-of-forwarding PACKET
send(p, dest) ← send message p to next hop dest
if S_id ∈ this.getNeighborList() then
    send(msg, S_id);
else
    record(< b_s, rnd >);
    NextHop = this.getNeighbor(b_s);
    send(msg, NextHop);
```

Claim 2. *Denote a participant S sends loop messages with an expected detection probability τ, malicious active attackers selectively drop messages with probability β and there are totally m messages being sent out in round rnd. Let $O_{m,rnd}$ be the scenario where S successfully detects misbehaves. Then,*

$$Pr\,(O_{m,rnd}) = 1 - \frac{[(1-\tau)\cdot m]! \cdot [(1-\beta)\cdot m]!}{m! \cdot [(1-\tau-\beta)\cdot m]!} \quad (3)$$

We refer to Appendix A for the proof. We conclude that DAENet can completely resist active message drops when adversaries entirely block the traffic from a participant. Also, DAENet probabilistically defend against selective message drops under active attacks according to an expected detection probability chosen by the participant.

4.4 Anti-Traceback: Keep Sender Identifier Confidential

Straw man approach. As discussed in §3.3.3, traceback attackers might collaborate with a compromised receiver in DAENet to reveal the identifier of a targeted sender by dropping or delaying messages. A straw man approach is to detect malicious traceback behaviors in the network. However, detecting traceback attacks in the network is difficult. First, naively setting a threshold ϵ as *time-out* at sender to cut off the communication is impractical because we cannot determine an average latency of communication in the network as the number of participants grows and network environment differs in places. If ϵ is too large, the detection threshold is useless; Otherwise, if ϵ is too small, communications become hard to carry on in the network. Second, *loop message detection* does *not* work in this scenario. *Loop* message is used to prove to a participant that a potentially withdrawn neighbor is online. However, since the sender does not know the exact or even relative position of the malicious receiver in the network, *loop* messages cannot tell whether the message drop is due to an offline receiver or a malicious traceback attacker.

To solve this problem, we utilize a *round-based dead drop* design to prevent malicious receivers from revealing the identifier of senders. The basic idea of this design is randomly selecting a sequence of participants in DAENet as destinations for two session nodes to exchange information in several *rounds*, and enabling full asynchrony to hide messaging patterns. Next, we discuss our *round-based dead drop* design and how we use SGX to hide the access pattern of dead drop nodes.

Round-based Dead Drop Messaging. DAENet enforces communications through a sequence of dead drop nodes. Dead drop nodes are virtual locations where two session nodes deposit their messages (*original messages*), swaps message payload from the same conversation and fetch messages (swapped messages) back. To initialize a conversation, two participants first negotiate a randomly generated *shared secret*. The *shared secret* is used for generating a sequence of *DeadDrop_keys*. The *DeadDrop_keys* are deterministically mapped to a set of nodes in the network.

Two session nodes (namely Alice and Bob) communicate with each other through these dead drop nodes. Communications happen in *rounds*. In round i, Alice and Bob independently send a message to a dead drop node N_i which is mapped from $DeadDrop_key_i$. Each message is labeled with a *session-round* pair to indicate its unique session identity with another participant and the round of payload exchange. When N_i receives a message m, it stores it and waits for the coming of m_1 which has the same *session-round* pair as m. When m_1 arrives, N_i swaps the payload of these two messages and sends them back to corresponding message originators. The round-based dead drop messaging is effective to defend against traceback attacks because communication circuits between session nodes changes with different dead drop nodes as destinations. Moreover, since Alice and Bob generate $DeadDrop_keys$ in SGX enclaves without involving an untrusted third-party, the locations of dead drop nodes used in the communication are kept confidential to other participants. Thus, traceback attackers cannot identify which nodes are chosen as dead drops to exchange the payload between Alice and Bob, making the communication circuit unpredictable. By splitting the static routing circuit into multiple unpredictable circuits, traceback attackers who keep monitoring the traffic cannot reveal the previous hop in a fixed circuit by dropping messages and observing the arrival of messages.

Note that compromised dead drop nodes could fail to execute the payload exchange and claim to be temporarily offline. DAENet tolerates dead drop node failures with a *switch* strategy. The core idea of the strategy is that session nodes do *not* need to wait for a successfully exchanged reply from dead drop nodes in each communication round. If Alice's message m was not sent back by dead drop node N_i, Alice can resend m by switching to another *unused* dead drop node N_j with reference to $DeadDrop_key_j$. DAENet provides such flexibility because DAENet supports reliable datagram transfer, rather than online streaming that needs ordered messages. Thus, we assume participants can tolerate a reasonable delay of some messages and transfer other messages first when a portion of dead drop nodes fail. In the worst case when all dead drop nodes mapped from $DeadDrop_keys$ are compromised, no successful payload exchange will take place. Since the dead drop nodes are randomly selected, the failure of all dead drop nodes indicates a potential monitoring of the network. Such vulnerability will be quickly detected and participants are suggested to carry on their conversations later.

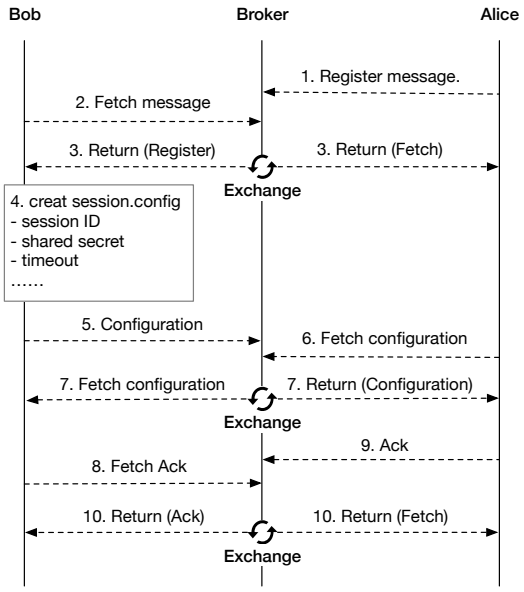


Figure 3: Two participants of DAENet initiates their conversation through a secure dialing protocol.

In addition to the *switch* strategy, to achieve privacy even with malicious dead drop nodes, DAENet leverages three policies listed as follows.

Trusted swapping. All dead drop behaviors are executed within SGX. Since SGX guarantees the confidentiality of decrypted messages in memory, a malicious dead drop node cannot determine whether two messages belong to the same conversation and which communication pairs are being swapped.

Ephemeral duty. Duties in DAENet are ephemeral which means the dead drop role do not need to persist over time. As DAENet works in asynchronous rounds, a dead drop node (agreed on by two participants) is only responsible for handling message swap in current communication round, unless being chosen by the two participants again. Hence, a malicious dead drop node will not always hold the conversation and have no chance to reveal the link between the two participants.

4.5 Secure Dialing: Conversation Initialization

DAENet uses a secure dialing protocol to help participants initialize anonymous conversations without leaking private information (e.g., identities of participants). Preventing private information leakage during the initialization process is important because a service provider (namely sp) may want to keep anonymous in the network and hide its identifier from the public. If sp's identifier is public, it may become the target of DoS attacks: sp's competitors can continuously send dummy messages to sp to block its service from other benign participants.

The dialing protocol involves three parties. The first party is a *client* c who wants to start a conversation with

another participant in DAENet's network. The second party is a *service provider sp* who provides services (e.g., secret file sharing) to participants. Since a service provider can provide many services, a service provider typically maintains a set of *service_keys*. A service key SK_{ij} denotes the ith service provided by a service provider sp_j. The last party is a *broker* node b_j which is a designated virtual location that is responsible for receiving conversation requests to a service provider sp_j in the network.

A typical application of DAENet is anonymous file-sharing where c tries to fetch a secret file from sp_j. Since sp_j has to hide its identifer and be reachable to others, we use a special dead drop node - the *broker* c to anonymously initialize conversation details without involving direct interactions between c and sp_j. The initialization mainly negotiates for three items: a shared secret sec, session ID s_id and an expiry time exp. sec is the seed of a pseudo random number generator. With the same sec, c and sp agree on the same set of dead drop nodes to exchange message payload in each communication round. s_id is the unique identity of the conversation which is used for dead drop nodes to identify awaiting messages from the same conversation, and exp is the longest duration for waiting a reply (i.e., time-out).

Figure 3 shows a complete procedure for the dialing protocol. Next, we introduce the steps from the perspective of the client and the service provider respectively.

For client: To fetch the ith service from service provider sp_j, the client c first finds the service key SK_{ij} from an external source. The source could be a database where DAENet's service providers put its service keys on. The client c negotiates conversation configurations with sp_j' by sending a *Register* message to sp_j's broker node b_j. The *Register* message contains the service key SK_{ij} to indicate c's requested service. The broker node b_j receives the message and verifies the contained service key to ensure a valid connection request from c. A service provider periodically asks its broker node whether there exists any *Register* messages. When sp_j finds out c's request for its ith service, it sends a configuration file to its broker node b_j. In next round, c sends a fetch request to b_j to fetch sp_j's configuration file. When c receives the configuration file, it sends an ACK to b_j to confirm a successful dialing process. By this step, the dialing process for a client is completed successfully.

For service provider: As a service provider, sp_j has two jobs: (1) securely assign a broker node to handle its initialization requests and (2) keep fetching initialization requests from its broker node and negotiating configuration files with clients. To complete the first job, sp_j sends a *endorsement* request to a random participant in the network in order to register for a broker service. If the participant replies with an acceptance, sp_j encapsulates a message which contains all the service keys it provides, and sends that message to the participant. The participant then serves as the broker node b_j to handle initialization requests. To complete the second job, sp_j periodically asks b_j if there exists any initialization requests from clients. If sp_j finds any service requests, it will send corresponding configuration file to its b_j, and b_j will send the configuration file to the client. Further, sp_j tries to fetch an ACK from its broker node, if sp_j receives an ACK, then the dialing process is completed.

With the help of a broker node, a client registers itself to a service provider without knowing the identify of the service provider, and the service provider can securely broadcast its services and receive conversation initialization requests from the network. With a negotiated configuration file for transmission, the client and service provider can further carry out communications.

5 Security Analysis

5.1 Analysis of Global Passive Attacks

In this subsection, we first give a theoretical proof of DAENet's oblivious messaging pattern that makes two participants in one conversation unlinkable, and then conduct a experiment to test the unlinkability under global passive attacks with a metrics *likelihood*.

5.1.1 Theoretical Proof of Oblivious Messaging

DAENet requires a participant to send messages to all its neighbors with the same probability because a biased messaging pattern can reveal sensitive information to global passive attackers. Next we prove that a DAENet participant sends messages to all its neighbors with the same probability and thus achieves full randomness.

Proof of Oblivious Messaging. Suppose that each node in the underlying Chord identifier ring $N <= 2^n - 1$ sends message to a random node in the ring. Each node id has $log_2 N$ neighbors, namely $id+2^i$ for each $i <= n$. Then each neighbor of an arbitrary node id has the same expectation on access time.

Claim 3. *Each neighbor of an arbitrary node id, denoted as $id + 2^i (0 <= i < n)$, has the same number of access.*

Proof. Suppose that two node x and y are two identical nodes in the ring, we evaluate one node id, where

$$x \to ... \to id \to ... \to y. \tag{4}$$

and the identifier of node x may be equal to id.

As the routing from x to y will pass id to y, then we have $x = id-(\sum_{i=0}^{n} X_i 2^i)$, and $y = id+(\sum_{i=0}^{n} Y_i 2^i)$. X_i and Y_i is a selection variable. If a message is routing from x to y, and passes id to its i_0^{th} neighbor of node id, then $X_i = 0, i <= i_0$ and $Y_i = 0, i > i_0$. Therefore, for a neighbour $id+2^i$ of node id, the total number of (x, y) pair that passes id and $id + 2^i$ is

$$(\sum_{k=0}^{n-1-i} C_{n-1-i}^{k})(\sum_{k=0}^{i} C_i^k) = 2^{n-1-i} 2^i = 2^{n-1} \tag{5}$$

, which is identical to all id's neighbors. □

Limited Observable Variables. With the full randomness proved above, DAENet's protocol reveals only a small, yet insensitive set of variables to global passive attackers. First, DAENet's shuffling protocol, used for hiding communication circuits, makes all participants run in a stealthy P2P network and exposes just two variables to adversaries: the total number of sent-out messages in each round and the output rate of participants. These two variables are insensitive because they cannot reveal which participant is actually talking, as adversaries cannot distinguish an application

message. Also, since we achieve full randomness of sending messages, observing the output rate does not reveal any sensitive information as well.

Second, by running code inside SGX, we prevent adversaries from directly intervening the protocol execution and seeing the decrypted plaintext of messages. Malicious participants can monitor traffic links and deduce a set of participants' predecessors and successors under DAENet's Chord topology. However, adversaries cannot distinguish whether a received message from a predecessor is a control message - used to maintain the network structure, or an application message - used for communication.

5.1.2 Experimental Proof of Defending Traffic Analysis

This subsection gives an end-to-end anonymity evaluation to analyze the impact of global passive attacks in DAENet. As the strongest traffic analyzer, GPAs monitor global traffic and observe messages entering and exiting a participant, in order to link corresponding message sender and receiver.

Thus, we analyze the unlinkability between senders and receivers by using an empirical analysis tool, used by Loopix, to study the correlation probability of two messages in the network. The security metrics that we use is called *likelihood difference*, which reveals the probability of linking a leaving message to a sender S_0 in comparison to another sender S_1. Denote the *likelihood difference* as ϵ, the two probabilities that a message is sent by S_0 and S_1 as $p_0 = Pr[S_0]$ and $p_1 = Pr[S_1]$. Our evaluated *likelihood difference* is

$$\epsilon = | \, log(\, p_0 \, / \, p_1 \,) \, | \tag{6}$$

in which p_0 and p_1 can be calculated from Equation (3) and Equation (2).

To study the probabilities, we run DAENet in a local cluster, ranging from 1,024 participants to 8,192 participants that generate and send messages simultaneously with unified messaging rate 50ms. Among the participants, 10% participants hold on communications while the left 90% participants do not communicate. We challenge the two senders S_0 and S_1 to analyze the probability: First, all participants wait for a membership warm-up time until the network becomes steady to test. All the 10% communication holders, except for S_0 and S_1, simultaneously send messages to the network. Then, let S_0 and S_1 encapsulate two messages, tag the two messages and send them to the network as well.

Now that there are two messages sent by S_0 and S_1 in the network which are manually labeled, while the remaining messages sent by other participants are not labeled. At each hop, we track the probability that an exiting message is labeled S_0 or S_1, and calculate the probability of being one of the senders through Theorem 2 (§4.2). As we pick S_0 and S_1 in their final destination, we calculate ϵ in Equation (6).

Varying the parameter of message emitting rate and shuffle rate, we average the evaluation results over 1000 repetitions and illustrate them in Figure 4 and Figure 5. Our experiment shows that the expected *likelihood difference* is small (lower than 0.31).

More participants, stronger anonymity. As we can see from Figure 4, ϵ degrades almost linearly with more participants. This indicates that, by increasing the number of users of

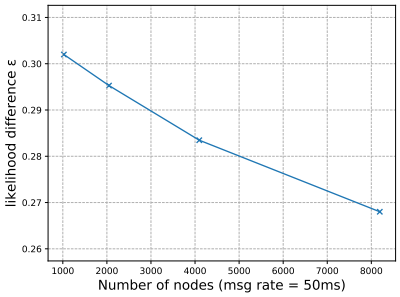


Figure 4: Likelihood difference ϵ depending on the number of participants in the network.

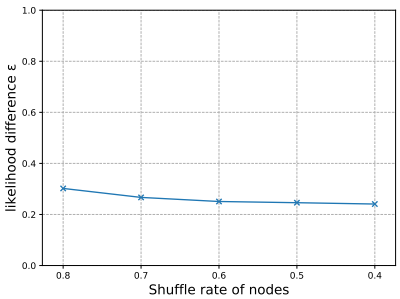


Figure 5: Likelihood difference ϵ depending on the shuffle rate for each participant in the network.

DAENet, the anonymity of participants can be further improved. When DAENet scales out to a large number of users, participants in the network process more messages. As all the messages fully mixed in shuffle pools, the likelihood difference of two senders decreases, indicating that GPAs have less probability to link message senders and receivers.

Parameter selection. Figure 5 shows that the expected *likelihood difference* decreases (0.30199 to 0.2409) with decreasing shuffle rate. This figure illustrates that (1) decreasing the probability of pulling a message from shuffle pools (by decreasing the shuffle rate) with respect to the message emitting rate increases anonymity and (2) the shuffle rate has small impact on the anonymity of participants. As the shuffler rate decreases, DAENet requires participants to send more dummy messages. To save the bandwidth cost, we consider shuffle rate = 0.8 to be a good choice in terms of anonymity.

Comparison with Loopix. Loopix also uses likelihood to evaluate its defending capability against global traffic attacks. Even if Loopix's likelihood can be smaller than DAENet, it incurs additional delay in each mix node. In Loopix's likelihood evaluation setup (i.e., a topology of 3 layers with 3 mix nodes per layer), when Loopix achieves comparable likelihood as DAENet (0.25), it incurs additional 1s delay in each mix node. Thus, Loopix sacrifices 3s latency throughout all three layers of shuffles which is larger than DAENet's end-to-end communication latency (see §6).

5.2 Analysis of Active Attacks

In this subsection, we analyze the impact of active attacks in the presence of DAENet. First, we analyze DAENet against

targeted DoS attacks when network attackers compromise a fraction of nodes to increase the chances of participants choosing fully malicious messaging path. We continue by evaluating the round-based dead drop design and the security of loop messages.

5.2.1 Resisting Targeted DoS Attacks

DAENet can defend against targeted DoS attacks with the assumption that the messages will not be relayed via a fully malicious routing circuit, which is entirely controlled by the adversary. If a routing circuit is fully controlled, the adversary can trivially track all traffic and deduce that the sender and receiver are within a small anonymity set. In other words, the sender will be one of the predecessors of the entry node of the circuit, and the receiver is considered to be one of the successors of the exit node of the circuit.

Because routing circuits are chosen by the underlying P2P lookup protocol, which is correctly executed within SGX, the only way the adversary can succeed in conducting targeted DoS attack is by adding more compromised nodes. However, the adversary cannot disconnect any honest nodes from being picked in a circuit because the loop messages periodically check the liveness of honest nodes to prevent them from being withdrawn, therefore, the adversary can only increase the probability of being chosen as a relay in circuit.

Denote M_{adv} as the set of compromised nodes controlled by the adversary, N as the total number of nodes in the network and p_m as the proportion of compromised nodes. During the circuit generation process, the probability of choosing a fully malicious routing circuit is

$$Pr(circuit \in M_{adv}) \leq (p_m)^{LogN} \tag{7}$$

Equation (7) indicates that adding more compromised nodes only slightly increases the probability of choosing a fully malicious routing circuit. When the network scales to 10,000 participants, even with a large compromised rate (p_{m1} = 0.8, p_{m2} = 0.5), the probability of successfully conducting targeted DoS is less than 0.05 and 0.0001, respectively. Even with a fully controlled routing circuit, the adversary still cannot distinguish whether a participant is talking to someone else or not. To further de-anonymize a message sender and receiver, the adversary has to sabotage the entry and exit traffic and collaborate with one party to ensure a conversation indeed happens through this fully compromised circuit. Note that the larger the scale of DAENet, the more expensive the targeted DoS attack, and the lower the probability of success.

5.2.2 Resisting Aggresive Active Attacks

As discussed in Section §4, active attackers can halt the anonymity through dropping and delaying messages, or even collaborate with DAENet participants to launch stronger attacks in the network. Active attackers might try other approaches to subvert the anonymity of participants. We first discuss the security of our approaches against most common and critical active attacks, and then discuss other potential attacks that try to de-anonymize DAENet participants.

Security of Loop Messages. A compromised participant might refuse to receive or forward predecessors' messages. If the dropped messages are application messages, such attack is imprudent because participants can simply wait for a short time and send that message in other communication round, without leaking anonymity; Otherwise, if control messages are dropped, such misbehaves will be detected by loop messages. The preceding participant can force the succeeding participant to formulate a *Proof-of-Forwarding* with a receipt to prove itself forwarded its messages.

Loop messages are secure against counterfeiting. Since the receipt must contain a secret key, which is generated by the the loop message sender and kept secret within SGX, compromised participants and network adversaries cannot duplicate that key to counterfeit the forwarding behavior. Also, the adversary is incapable to determine a loop message. Because all the loop messages are encapsulated in the same way as genuine messages, and processed through the shuffling. Thus, adversaries cannot seek to predict a loop message by observing the timing pattern.

Defeating DAENet Anti-Traceback Protocol. DAENet uses round-based dead drop design to protect senders/receivers from being observed by active attackers who collaborate with compromised participants. As we discussed in §3.3, a fixed circuit in a P2P network gives chances to attackers to hierarchically reconstruct the message path. By using the dialing protocol (§4.5) to agree on a set of dead drop nodes in the network, DAENet prohibits adversaries from tracking an honest participant and revealing its identity. We trust the PRNG used in the dialing protocol to generate a series of $DeadDrop_keys$, adversaries cannot predict the next dead drop node to exchange message payload.

Therefore, defeating DAENet's anti-traceback protocol requires active attackers precisely delay all the on-path application messages in each communication round. Denote the normal averaged end-to-end communication latency as l_1, the delay time as T_d and the expected path length through dead drop as l_2. As the expected communication time through dead drop nodes is fixed, if the traceback attacker receives a delayed message whose latency is $l_1 + (T_d \times l_2)$, then she might have confidence to reveal the message sender.

However, precisely blocking all the on-path message is difficult, meanwhile, the usable time is short. As we will show in §6.1, the expectation of averaged end-to-end latency is less than $2.2s$. To successfully defeat the protocol, the adversary has to precisely predict and delay all the on-path application messages with probability $1/logN$ for each link within $2.2s$, where N is the total number of participants. Therefore, this attack is beyond the capability of attackers.

Aggresive Hijacking Packets. To de-anonymize network participants, a more aggressive approach is to drop a significant number of messages. For example, active attackers can launch $(n-1)$ attack [50] to track a specific message from Alice by blocking other messages to an honest participant. Also, network adversaries can inject malformed messages to replace ordinary messages. Note that in this scenario, an honest participant can easily detect such misbehavior and notice a compromised successor in the network. Honest participants can simply rejoin the network to switch to a new location and fetch a new list of neighbors for anonymous messaging.

Traffic Watermarking Attacks. Pointed out by Xinyuan [51],

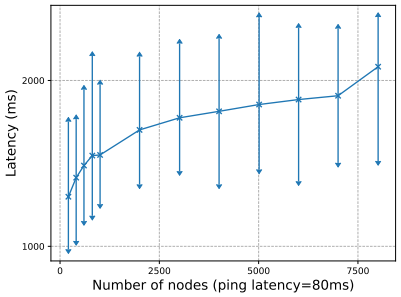


Figure 6: Latency of DAENet when 50 to 8000 participants simultaneously send traffic at rate $\mu = 50ms$ and shuffle messages with probability $\delta = 0.8$. We assume that there is no additional delay add by participants.

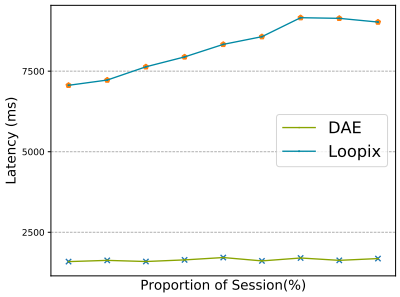


Figure 7: Latency of DAENet for anonymous communicating for varying number of sessions. The latency does *not* increases as the number of session grows.

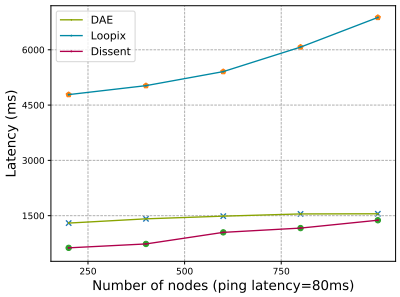


Figure 8: Latency comparison. We measured Loopix and Dissent - two state-of-art scalable anonymous messaging systems.

many proposed low-latency anonymous communication systems are vulnerable to traffic watermarking attacks. In the attack, a compromised service provider tags watermarks at messages from suspected clients, and determines if the suspected client visited the service by checking if that user has received the watermarked traffic. DAENet can defend against traffic watermarking attacks because (1) DAENet's anonymous traffic flow and the application traffic flow is mixed by trustworthy message shuffles and fully random dissemination from participants to their neighbors. Thus, a watermarking attacker cannot precisely tag an application message. (2) Even if watermarking attackers can tag application messages, they cannot reveal clients because clients are not the destinations in each round of communication, instead attackers can only reveal the set of randomly selected dead drop nodes for exchanging messages.

Under attack or Network Congestion? One possible question in DAENet is how to differentiate between message dropping due to a compromised participants or network congestion. In theory, both of them can make DAENet lose its liveness while malicious message drop may also lead to privacy leakage (shown in §4.4). In DAENet, it is not a critical issue to differentiate between these two circumstances because DAENet is not a penalty-based system (e.g., Miranda) that makes compromised participants lose their connections in the network. On the contrary, DAENet detects messages drops to maintain a consistent view of membership in the network, caused by either misbehaves or network congestion, thus honest participants will *not* be wrongly punished.

6 Evaluation

Our evaluation was conducted on 20 computers with SGX-equipped Intel(R) Xeon(R) CPU E3-1280 v6 with 24 cores, 64GB RAM and 2TB SSD. All computers form a cluster with 40Gbps network. In our cluster, each machine runs multiple (up to 400) instances of DAENet client. We used Linux Traffic Control (`TC`) to set the network latency between clients as $40ms$ to simulate the Internet environment.

We compared DAENet's performance with two state-of-art shuffle-based anonymous communication systems: Loopix and Dissent. Loopix is a popular open-sourced anonymous network that leverages poisson-mixing shuffle strategy to protect users in the same conversation from being observed by global passive attackers, which is also guaranteed by DAENet and has been proved in §5.1.1. We also compared DAENet's performance with Dissent. Dissent is another open-sourced anonymous network that leverages verifiable shuffles to defend against global passive attacks. Although Dissent suffers from long-term active intersection attacks [52], it is well-known for its support of low-latency communications compared to other shuffle-based systems (e.g., Riposte, Atom). Other shuffle-based systems such as Karaoke and Vuvuzela are not evaluated because they are not open-sourced.

We built an anonymous chatting application to evaluate the performance of DAENet and our baseline systems. In our chatting application, two participants communicate with each other by sending close-loop messages through a set of dead drop nodes. To match the real-world workload of online communications, we sampled X% of all participants as active message senders while other participants still work as normal relays. The ratio X% is set to 10% by default, with reference to the Daily Active Users (DAU) of the popular WhatsApp application [53]. As Loopix has a slightly different architecture, we modified Loopix's code and wrote interfaces to forward the chatting traffic in Loopix's private cluster. Except for the client scalability evaluation, we run 50 clients on each machine (totally 1,000 clients) to evaluate the performance.

Our evaluation answers the following questions:

- §6.1 Can DAENet support a large number of participants and provide acceptable performance?
- §6.2 How sensitive is DAENet to its parameters?
- §6.3 How robust is DAENet to network churn and machine failure?

instance/machine	20	40	60	80	100
bandwidth/instance (MB/s)	0.14	0.14	0.14	0.14	0.13

Table 2: Bandwidth cost of running DAENet.

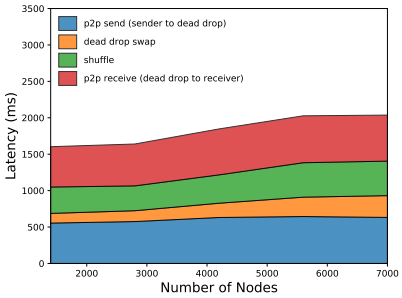


Figure 9: Breakdown of DAENet latency.

6.1 Efficiency and Scalability

To analyze the efficiency and scalability of DAENet, we answer the following three questions in this subsection:

- Can DAENet support a large number of users and scale horizontally?
- How does DAENet compare to prior systems?
- Will DAENet slow down the communication?

Horizontal scalability. To demonstrate that DAENet scales horizontally, we measured the end-to-end latency for participants to route million messages as the number of participants varied. As shown in Figure 6, the latency increases logarithmically with increasing number of users. When 8000 participants send traffic simultaneously the latency is nearly 2000ms.

Note that the latency overhead increases logarithmically with total number of participants. This is because the underlying topology of DAENet is a structured P2P network, where the expected path length for one *lookup* request grows logarithmically. In DAENet we utilize Chord, the expected path length for a lookup is $log\mathcal{N}$, where $\mathcal{N}$ is the total number of participants in the network. When DAENet scales to 1M participants, the expected path length for a lookup only grows to 20.

Number of messages. To evaluate how the number of active nodes affects latency, we increased the proportion of active nodes (i.e., nodes in communication sessions) from 10% to 95%, and measured the network latency, as shown in Figure 7. As the proportion of active node increases, DAENet's latency does not increase much, while Loopix's increases dramatically. This is because Loopix incurs larger shuffle overhead with a growing number of messages through its centralized mix servers. On the contrary, participants in DAENet still send dummy messages even if there is no application messages to send, hence increasing the portion of active nodes does not produce additional network overhead because the previous idle participants just change a kind of emitted messages.

Comparison to prior work. To compare DAENet's scalability we ran an experiment in our cluster with 20 servers. To evaluate the support for growing participants, we simulated clients by running multiple ($10 \sim 600$) instances on each machine. For comparison, we also include the latency of Loopix and Dissent as reported in previous subsection which are the only *open-sourced* anonymous messaging system that claims to be scalable to users. We picked the system parameters $\mu = 50ms$ as the message emitting rate of participants in the network, and $\delta = 0.8$ as the shuffle rate to mix real messages and dummy messages.

Figure 8 shows that with 800 users DAENet achieves $1.5X$ higher latency than Dissent, and $5X$ lower latency compared to Loopix. The reason why DAENet incurs higher latency than Dissent is that Dissent is a centralized system and it statically assigns servers for clients to send their messages, thus clients in Dissent doesn't need to forward messages through several hops and save the time for lookups. However, such design exposes attack surface to DoS all the static servers. DAENet scales better than Loopix because all Loopix traffic must go through a single chain of servers while DAENet requires each participant to only process a fraction of messages in the network.

Latency Breakdown. To investigate DAENet's latency, we break down DAENet's latency incurred by shuffle, dead drop messaging and P2P communications, as shown in Figure 9. Around 69.1% of the latency is from P2P communication, as it requires $log(N)$ steps to locate a node in the network. Dead drop communication contributes 8.4% of the latency. The last source of the latency, message shuffling, incurs only 22.5% of the latency.

As we can see from the breakdown results, DAENet will slow down the communication by adding 30.9% more round-trip latency. However, we believe that DAENet is useful for anonymous online communications, as participants may value stronger privacy guarantee and tolerate the moderate latency.

Bandwidth Usage. We test the bandwidth usage in a cluster of 14 machine where each machine holds several instances running independent DAENet protocol. Table 2 shows the bandwidth usage of participants running DAENet protocol. In this experiment, each test has one conversation between two randomly picked participants from all instances. To understand the minor bandwidth cost (around 0.14MB/s), DAENet's design crucially avoids heavy usage of network resource for sending dummy messages. This is because we also add P2P control messages to the shuffle pools, so that when participants have to send out a dummy message to a neighbor, it can just replace by sending a control message rather than a dummy message. The sending of control messages in DAENet follows the rules of Chord, each participant refreshes its view of membership by sending control messages to all its neighbors every 1 second.

6.2 Parameter Sensitivity

To understand how the parameters (i.e., shuffle rate and message emitting rate) affects latency, we varied the minimum shuffle rate and message emitting rate, and measured the latency, as shown in Figure 10 and Table 3. With unified message emitting rate $50ms$, the latency increases dramatically when shuffle rate is decreased. This is because, in each shuffle pool of a neighbor, with a smaller shuffle rate, the probability of popping out a real message to that neighbor becomes smaller and the probability of sending a dummy message to that neighbor becomes larger. That

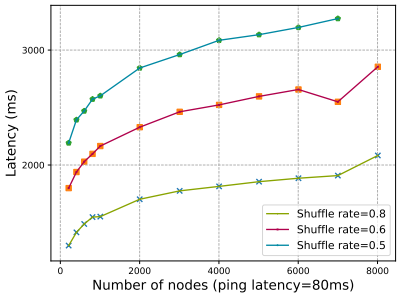


Figure 10: The end-to-end latency of DAENet with unified message emitting rate 50ms and varying shuffle rate.

Message emit rate (μs)	100	500	1000	2000	3000	5000
Latency (ms)	773	758	787	1055	1302	1594

Table 3: The end-to-end latency of DAENet with varying message emitting rate, running in a cluster of 1000 nodes.

is, a real message will have less chance to be sent out to its destination and the latency increases. Note that with a smaller shuffle rate, DAENet guarantees more obliviousness of output messages, since real messages are fully mixed with dummy messages and a malicious observer is more difficult to distinguish a real message.

When the message emitting rate increases, the latency of messages decreases because a message is popped out of the shuffle pool more quickly with larger emitting rate. However, the descending trend of latency is smoother with large emitting rate. This is because that the latency is also bounded by dead drop swap and P2P communication.

6.3 Failure Recovery

Handling node churn is a major issue in P2P systems. To evaluate the failure resilience of DAENet, we ran DAENet for a period of time, with a typical message emitting rate $50ms$ and shuffle rate 0.8, and we arbitrarily killed 10% of all participants for three times (totally killed 30% active participants). The killed nodes of DAENet are sampled uniformly from existing participants, including both active participants (communicating) and idle participants. Figure 11 shows the latency before and after killing nodes. When nodes are killed, DAENet's latency becomes extremely high because the lost of transferring message triggers timeout. After that, DAENet's latency resumes to normal in a short time, as DAENet detects failure of message, updates routing table and resumes processing.

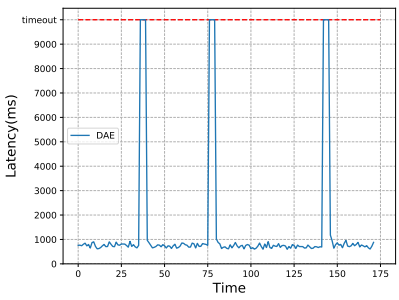


Figure 11: Arbitrarily killing DAENet nodes to simulate network churn with a 10% killing rate.

7 Discussion

DAENet has two limitations. First, current DAENet implementation do not integrate side-channel attack defenses. As SGX is susceptible to side-channel attacks where malicious software on the same platform can infer enclave data access patterns by monitoring shared resources such as caches [54], [55], it is fixable by using well-known Oblivious Ram (ORAM) algorithms, such as ZeroTrace [56].

Second, DAENet currently only supports point-to-point anonymous communication rather than anonymous broadcast, in which a participant can broadcast items to a set of receivers in an anonymous manner. This limitation forbids DAENet from supporting some security-sensitive broadcast applications such as the transaction dissemination in Bitcoin P2P network. Supporting anonymous broadcast in a P2P network could be an interesting future direction of DAENet.

8 Related Work

Trusted Execution Environment. TEE provides strong security guarantees (i.e., confidentiality and integrity) for applications with efficiency. Intel SGX [57] is one of the most popular TEE in the market. With the convenience and security properties introduced by SGX, it has been adopted for secure data analysis [45], [58], network analysis [59] and secure key-value stores [60].

SGX-Tor [61] is the first work that applies SGX to anonymous network. As the first SGX enabled anonymous network, SGX-Tor proves the feasibility of running SGX-enabled hosts to improve an anonymous communication system's security model. As Tor relays are under the control of world-wide users, running the Tor protocol inside SGX effectively prevents malicious Tor relays from gaining private information of Tor components, such as circuit identifiers and hidden service identifiers. Although SGX-Tor mitigates many attacks against malicious Tor components, it cannot defend against network-level adversaries such as global passive attackers and active attackers, potentially preventing it from being a choice of users who value strong privacy. DAENet also leverages SGX to prevent private information leakage and regulate participants' behaviors. Moreover, we improves SGX-Tor's security model by protecting participants from global passive attacks and active attacks that maliciously drop and delay messages. Although DAENet incurs slightly higher end-to-end communication latency compared to SGX-Tor (shown in Table 1), we believe that users may tolerate DAENet's moderate latency to achieve stronger privacy guarantee in anonymous communication.

Relay-based Anonymous Network. The most popular relay-based anonymous communication system is Tor [15]. Due to its popularity and transparent development processes [62], many researchers have explored attacks that can de-anonymize Tor users and hidden-service providers by monitoring the network traffic. Recent attack directions for Tor include website fingerprinting [63], [64], [65], [66], traffic correlation [10], [11], [12], [67], congestion attack [68], [69] and targeted DoS [39], [70], [71]. Meanwhile, researchers

also propose methods to enhance Tor's security by optimizing the bandwidth report for selecting guard nodes [72] and monitoring circuit construction [73]. Also, some recent Tor improvements consider generating cover traffic within middle routers of circuits, such that the middle routers can hide any relationship between compromised entry and exit nodes [74], [75]. To Recently, SGX-Tor [33] explores a new direction to enhance Tor's anonymity by protecting sensitive metadata within trusted hardware (e.g., Intel SGX [76]). ShadowWalker [34] and AP3 [35] are built upon a structured P2P network to eliminate the concern of targeted DoS attacks. However, these systems are not designed for achieving strong anonymity guarantees against both passive and active traffic analysis attacks.

Shuffle-based Anonymous Network. Shuffle-based approaches are promising to defend against strong global passive attackers. The most popular shuffle strategy *onion mixnet* is leveraged by numerous anonymous communication systems including Loopix [18], Atom [36], Vuvuzela [77] and Karaoke [20]. An onion mixnet encrypts a sender message multiple times by using the public keys of all on-path mix servers, and shuffles messages at each mix server to mess up the order of input and output messages, such that a network eavesdropper cannot distinguish the real routing circuit of a particular message. Unfortunately, such design typically has several simplifying assumptions: **(1)** a stable network environment where operations are executed in synchronized batches, **(2)** a fixed set of mix servers and a majority of them are benign. It's non-trivial to protect shuffle-based systems from active attacks while maintaining acceptable end-to-end communication latency. For example, while Riposte [21] handles active attacks, it assumes that clients are willing to tolerate hours of messaging latency. Dissent [29] provides low-latency communication but it reports poor fault-tolerance under active attacks: if a server goes offline, Dissent's protocol completely halts until all servers are available again, which severely limits Dissent's practicability.

9 Conclusion

DAENet is the first work that enables strong anonymity in a fully decentralized network. DAENet provides both sender-receiver unlinkability under global passive attacks, and sender/receiver anonymity under active attacks. DAENet is built upon a structural p2p network for efficiency and adopts sophisticated techniques, including the dead drop abstraction and shuffling for defending traffic analysis. DAENet makes use of Intel SGX to protect the integrity and confidentiality of the structural network and message shuffles. DAENet's evaluation shows that it scales well with moderate end-to-end latency.

Appendix A
Proof of Claim 2

Recall that there are totally m messages sent in round rnd. With an expected detection probability τ, there are totally $\lceil \tau \cdot m \rceil$ loop messages contained in the m messages. With a message drop rate β, $\lceil \beta \cdot m \rceil$ messages are dropped in round rnd. If one out of all loop messages is dropped, a participant can detect the malicious message drop. Thus, for each communication round,

$$\begin{aligned} Pr(detectable) &= Pr(\exists\ dropped\ loop\ message) \\ &= 1 - Pr(\forall\ loop\ messages\ are\ not\ dropped) \\ &= 1 - \frac{C_{(1-\tau)\cdot m}^{\beta \cdot m}}{C_m^{\beta \cdot m}} \\ &= 1 - \frac{[(1-\tau)\cdot m]!}{(\beta \cdot m)! \cdot [(1-\tau-\beta)\cdot m]!} \\ &\cdot \frac{(\beta \cdot m)! \cdot [(1-\beta)\cdot m]!}{m!} \\ &= 1 - \frac{[(1-\tau)\cdot m]! \cdot [(1-\beta)\cdot m]!}{m! \cdot [(1-\tau-\beta)\cdot m]!} \end{aligned} \tag{8}$$

This concludes our proof.

References

[1] A. Johnson, C. Wacek, R. Jansen, M. Sherr, and P. Syverson, "Users get routed: Traffic correlation on tor by realistic adversaries," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. ACM, 2013, pp. 337–348.

[2] B. Resch and J. Engdegård, "Metadata time marking information for indicating a section of an audio object," Aug. 11 2015, uS Patent 9,105,300.

[3] B. Harris and R. Hunt, "TCP/IP security threats and attack methods," *Computer communications*, vol. 22, no. 10, pp. 885–897, 1999.

[4] R. Jansen, M. Traudt, and N. Hopper, "Privacy-preserving dynamic learning of tor network traffic," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2018, pp. 1944–1961.

[5] R. W. Lai, K.-F. Cheung, S. S. Chow, and A. M. So, "Another look at anonymous communication," *IEEE Transactions on Dependable and Secure Computing*, 2018.

[6] E. Stoycheff, "Under surveillance: Examining facebooks spiral of silence effects in the wake of nsa internet monitoring," *Journalism & Mass Communication Quarterly*, vol. 93, no. 2, pp. 296–311, 2016.

[7] P. Winter and S. Lindskog, *How the great firewall of china is blocking tor*. USENIX-The Advanced Computing Systems Association, 2012.

[8] S. Saroiu, P. K. Gummadi, and S. D. Gribble, "Measurement study of peer-to-peer file sharing systems," in *Multimedia Computing and Networking 2002*, vol. 4673. International Society for Optics and Photonics, 2001, pp. 156–170.

[9] D. A. Bryan, B. B. Lowekamp, and C. Jennings, "Sosimple: A serverless, standards-based, p2p sip communication system," in *First International Workshop on Advanced Architectures and Algorithms for Internet Delivery and Applications (AAA-IDEA'05)*. IEEE, 2005, pp. 42–49.

[10] A. Johnson, C. Wacek, R. Jansen, M. Sherr, and P. Syverson, "Users get routed: Traffic correlation on tor by realistic adversaries," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 337–348.

[11] Z. Ling, J. Luo, W. Yu, X. Fu, D. Xuan, and W. Jia, "A new cell counter based attack against tor," in *Proceedings of the 16th ACM conference on Computer and communications security*, 2009, pp. 578–589.

[12] Z. Ling, J. Luo, W. Yu, X. Fu, W. Jia, and W. Zhao, "Protocol-level attacks against tor," *Computer Networks*, vol. 57, no. 4, pp. 869–886, 2013.

[13] S. J. Murdoch and G. Danezis, "Low-cost traffic analysis of tor," in *2005 IEEE Symposium on Security and Privacy (S&P'05)*. IEEE, 2005, pp. 183–195.

[14] G. Wang, F. Musau, S. Guo, and M. B. Abdullahi, "Neighbor similarity trust against sybil attack in p2p e-commerce," *IEEE transactions on parallel and distributed systems*, vol. 26, no. 3, pp. 824–833, 2014.

[15] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," Naval Research Lab Washington DC, Tech. Rep., 2004.

[16] S. J. Murdoch and G. Danezis, "Low-cost traffic analysis of tor," in *2005 IEEE Symposium on Security and Privacy (S&P'05)*. IEEE, 2005, pp. 183–195.

[17] J. Reardon and I. Goldberg, "Improving tor using a tcp-over-dtls tunnel." in *USENIX Security Symposium*, 2009, pp. 119–134.

[18] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, "The loopix anonymity system," in *26th USENIX Security Symposium USENIX Security 17)*, 2017, pp. 1199–1216.

[19] D. I. Wolinsky, H. Corrigan-Gibbs, B. Ford, and A. Johnson, "Dissent in numbers: Making strong anonymity scale," in *Presented as part of the 10th USENIX Symposium on Operating Systems Design and Implementation OSDI 12)*, 2012, pp. 179–182.

[20] D. Lazar, Y. Gilad, and N. Zeldovich, "Karaoke: Distributed private messaging immune to passive traffic analysis," in *13th USENIX Symposium on Operating Systems Design and Implementation OSDI 18)*, 2018, pp. 711–725.

[21] H. Corrigan-Gibbs, D. Boneh, and D. Mazières, "Riposte: An anonymous messaging system handling millions of users," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 321–338.

[22] H. Leibowitz, A. M. Piotrowska, G. Danezis, and A. Herzberg, "No right to remain silent: isolating malicious mixes," in *28th USENIX Security Symposium USENIX Security 19)*, 2019, pp. 1841–1858.

[23] D. Lazar, Y. Gilad, and N. Zeldovich, "Karaoke: Distributed private messaging immune to passive traffic analysis," in *13th USENIX Symposium on Operating Systems Design and Implementation OSDI 18)*, 2018, pp. 711–725.

[24] U. Parampalli, K. Ramchen, and V. Teague, "Efficiently shuffling in public," in *International Workshop on Public Key Cryptography*. Springer, 2012, pp. 431–448.

[25] J. Van Den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, "Vuvuzela: Scalable private messaging resistant to traffic analysis," in *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 2015, pp. 137–152.

[26] L. M. Sumitra and S. C. Miller, "Pathologic gambling disorder: How to help patients curb risky behavior when the future is at stake," *Postgraduate medicine*, vol. 118, no. 1, pp. 31–37, 2005.

[27] P. Paillier *et al.*, "Public-key cryptosystems based on composite degree residuosity classes," in *Eurocrypt*, vol. 99. Springer, 1999, pp. 223–238.

[28] S. Angel and S. Setty, "Unobservable communication over fully untrusted infrastructure," in *12th USENIX Symposium on Operating Systems Design and Implementation OSDI 16)*, 2016, pp. 551–569.

[29] D. I. Wolinsky, H. Corrigan-Gibbs, B. Ford, and A. Johnson, "Dissent in numbers: Making strong anonymity scale," in *Presented as part of the 10th USENIX Symposium on Operating Systems Design and Implementation OSDI 12)*, 2012, pp. 179–182.

[30] D. Chaum, "The dining cryptographers problem: Unconditional sender and recipient untraceability," *Journal of cryptology*, vol. 1, no. 1, pp. 65–75, 1988.

[31] G. Noubir and A. Sanatinia, "Trusted code execution on untrusted platforms using intel sgx," *Virus bulletin*, 2016.

[32] M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, "Malware guard extension: Using sgx to conceal cache attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2017, pp. 3–24.

[33] S. Kim, J. Han, J. Ha, T. Kim, and D. Han, "SGX-Tor: A secure and practical tor anonymity network with sgx enclaves," *IEEE/ACM Transactions on Networking*, vol. 26, no. 5, pp. 2174–2187, 2018.

[34] P. Mittal and N. Borisov, "Shadowwalker: peer-to-peer anonymous communication using redundant structured topologies," in *Proceedings of the 16th ACM conference on Computer and communications security*. ACM, 2009, pp. 161–172.

[35] A. Mislove, G. Oberoi, A. Post, C. Reis, P. Druschel, and D. S. Wallach, "AP3: Cooperative, decentralized anonymous communication," in *Proceedings of the 11th workshop on ACM SIGOPS European workshop*. ACM, 2004, p. 30.

[36] A. Kwon, H. Corrigan-Gibbs, S. Devadas, and B. Ford, "Atom: Horizontally scaling strong anonymity," in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 406–422.

[37] A. Mani, T. Wilson-Brown, R. Jansen, A. Johnson, and M. Sherr, "Understanding tor usage with privacy-preserving measurement," in *Proceedings of the Internet Measurement Conference 2018*, 2018, pp. 175–187.

[38] M. AlSabah and I. Goldberg, "Performance and security improvements for tor: A survey," *ACM Computing Surveys (CSUR)*, vol. 49, no. 2, pp. 1–36, 2016.

[39] R. Jansen, T. Vaidya, and M. Sherr, "Point break: A study of bandwidth denial-of-service attacks against tor," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 1823–1840.

[40] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, "Private information retrieval," in *Proceedings of IEEE 36th Annual Foundations of Computer Science*. IEEE, 1995, pp. 41–50.

[41] R. Ostrovsky and V. Shoup, "Private information storage," in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, 1997, pp. 294–303.

[42] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," *ACM SIGCOMM Computer Communication Review*, vol. 31, no. 4, pp. 149–160, 2001.

[43] J. Song and S. Wang, "The pastry algorithm based on dht." *Computer and Information Science*, vol. 2, no. 4, pp. 153–157, 2009.

[44] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for cpu based attestation and sealing," in *Proceedings of the 2nd international workshop on hardware and architectural support for security and privacy*, vol. 13. ACM New York, NY, USA, 2013.

[45] F. Schuster, M. Costa, C. Fournet, C. Gkantsidis, M. Peinado, G. Mainar-Ruiz, and M. Russinovich, "VC3: Trustworthy data analytics in the cloud using sgx," in *Security and Privacy (SP), 2015 IEEE Symposium on*. IEEE, 2015, pp. 38–54.

[46] H. Ballani, P. Francis, and X. Zhang, "A study of prefix hijacking and interception in the internet," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 4, pp. 265–276, 2007.

[47] Y. Gilad and A. Herzberg, "Spying in the dark: Tcp and tor traffic analysis," in *International symposium on privacy enhancing technologies symposium*. Springer, 2012, pp. 100–119.

[48] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the intel {SGX} kingdom with transient out-of-order execution," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 991–1008.

[49] O. Weisse, J. Van Bulck, M. Minkin, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, R. Strackx, T. F. Wenisch, and Y. Yarom, "Foreshadow-ng: Breaking the virtual memory abstraction with transient out-of-order execution," 2018.

[50] A. Serjantov, R. Dingledine, and P. Syverson, "From a trickle to a flood: Active attacks on several mix types," in *International Workshop on Information Hiding*. Springer, 2002, pp. 36–52.

[51] X. Wang, S. Chen, and S. Jajodia, "Network flow watermarking attack on low-latency anonymous communication systems," in *2007 IEEE Symposium on Security and Privacy (SP'07)*. IEEE, 2007, pp. 116–130.

[52] D. Kedogan, D. Agrawal, and S. Penz, "Limits of anonymity in open environments," in *International Workshop on Information Hiding*. Springer, 2002, pp. 53–69.

[53] C. Montag, K. Błaszkiewicz, R. Sariyska, B. Lachmann, I. Andone, B. Trendafilov, M. Eibes, and A. Markowetz, "Smartphone usage in the 21st century: who is active on whatsapp?" *BMC research notes*, vol. 8, no. 1, p. 331, 2015.

[54] F. Brasser, U. Müller, A. Dmitrienko, K. Kostiainen, S. Capkun, and A.-R. Sadeghi, "Software grand exposure:sgx cache attacks are practical," in *11th USENIX Workshop on Offensive Technologies (WOOT 17)*, 2017.

[55] J. Götzfried, M. Eckert, S. Schinzel, and T. Müller, "Cache attacks on intel sgx," in *Proceedings of the 10th European Workshop on Systems Security*, 2017, pp. 1–6.

[56] S. Sasy, S. Gorbunov, and C. W. Fletcher, "Zerotrace: Oblivious memory primitives from intel sgx." *IACR Cryptol. ePrint Arch.*, vol. 2017, p. 549, 2017.

[57] Intel, "Software guard extensions programming reference."

[58] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform." in *NSDI*, 2017, pp. 283–298.

[59] M.-W. Shih, M. Kumar, T. Kim, and A. Gavrilovska, "S-nfv: Securing nfv states by using sgx," in *Proceedings of the 2016 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization*. ACM, 2016, pp. 45–48.

[60] C. Priebe, K. Vaswani, and M. Costa, "Enclavedb: A secure database using sgx," in *EnclaveDB: A Secure Database using SGX*. IEEE, 2018, p. 0.

[61] S. M. Kim, J. Han, J. Ha, T. Kim, and D. Han, "Enhancing security and privacy of tor's ecosystem by using trusted execution environments." in *NSDI*, 2017, pp. 145–161.

[62] T. B. Tracker, "Wiki," 2018.
[63] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 605–616.
[64] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1187–1203.
[65] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier," in *Proceedings of the 2009 ACM workshop on Cloud computing security*, 2009, pp. 31–42.
[66] S. Li, H. Guo, and N. Hopper, "Measuring information leakage in website fingerprinting attacks and defenses," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1977–1992.
[67] S. J. Murdoch and P. Zieliński, "Sampled traffic analysis by internet-exchange-level adversaries," in *International workshop on privacy enhancing technologies*. Springer, 2007, pp. 167–183.
[68] N. S. Evans, R. Dingledine, and C. Grothoff, "A practical congestion attack on tor using long paths." in *USENIX Security Symposium*, 2009, pp. 33–50.
[69] J. Geddes, R. Jansen, and N. Hopper, "How low can you go: Balancing performance with anonymity in tor," in *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2013, pp. 164–184.
[70] R. Jansen, F. Tschorsch, A. Johnson, and B. Scheuermann, "The sniper attack: Anonymously deanonymizing and disabling the tor network," Office of Naval Research Arlington VA, Tech. Rep., 2014.
[71] N. Borisov, G. Danezis, P. Mittal, and P. Tabriz, "Denial of service or denial of security?" in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 92–102.
[72] R. Snader and N. Borisov, "A tune-up for tor: Improving security and performance in the tor network." in *NDSS*, vol. 8, 2008, p. 127.
[73] A. Panchenko, F. Lanze, and T. Engel, "Improving performance and anonymity in the tor network," in *2012 IEEE 31st International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2012, pp. 1–10.
[74] M. Soltani, S. Najafi, and R. Jalili, "Mid-defense: Mitigating protocol-level attacks in tor using indistinguishability obfuscation," in *2014 11th International ISC Conference on Information Security and Cryptology*. IEEE, 2014, pp. 214–219.
[75] M. Juárez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Wtf-pad: toward an efficient website fingerprinting defense for tor," *CoRR, abs/1512.00524*, 2015.
[76] V. Costan and S. Devadas, "Intel SGX explained." *IACR Cryptology ePrint Archive*, vol. 2016, p. 86, 2016.
[77] J. Van Den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, "Vuvuzela: Scalable private messaging resistant to traffic analysis," in *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 2015, pp. 137–152.



Tianxiang Shen is a PhD student supervised by Dr. Heming Cui in Computer Science Department at the University of Hong Kong. His research interests lie in distributed systems, with a particular focus on the system security and data privacy. Prior to his current program, he received B.Eng degree from JiLin University the Excellent Engineering Class.



Jianyu Jiang is currently a third year PhD student in Computer Science Department at The University of Hong Kong. He is working on topics in large scale computation platform under the supervision of Dr. Heming Cui. Prior to his current program, Jianyu receive his Bachelor's Degree in Computer Science Department at Xi'an Jiaotong University, under the supervision of Professor Qi Yong.



Yunpeng Jiang is currently pursuing his Bachelor degree in information security at Department of Computer Science and Engineering, South China University of Technology. His research interests includes system security and trusted computing tecnhiques.



Xusheng Chen received his Bachelor degree in HKU. He is currently a PhD student in Computer Science of HKU (2017-now). He is under the supervision of Dr. Heming Cui. His research interests include distributed consensus protocols, distributed systems and system security.



Ji Qi received his B.S (2015) degree from Beijing Institude of Technology, Beijing, China, and his M.S (2018) degree from Tsinghua University, Beijin, China. He is currently pursuing the PhD in computer science at the University of Hong Kong under the supervision of Dr. Heming Cui. His interests include domain-specific modeling, distributed system and cloud computing.



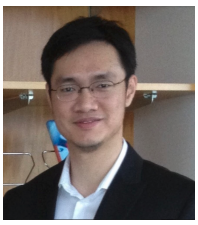
Shixiong Zhao received his Bachelor degree in HKU and his master degree in HKUST. He is currently a PhD student in Computer Science of HKU (2017-now). He is under the supervision of Dr. Heming Cui. His research interests include distributed systems for high performance computing, distributed systems and system security.



Fengwei Zhang is an Associate Professor at Department of Computer Science and Engineering at Southern University of Science and Technology (SUSTech). His primary research interests are in the areas of systems security, with a focus on trustworthy execution and hardware-assisted security. Before joining SUSTech, he spent four wonderful years as an Assistant Professor at Department of Computer Science at Wayne State University.



Xiapu Luo obtained his Ph.D. degree in Computer Science from the Hong Kong Polytechnic University, under the supervision of Prof. Rocky K.C. Chang. After that, he spent two years at the Georgia Institute of Technology as a postdoctoral research fellow advised by Prof. Wenke Lee. His research interests include Network and System Security, Information Privacy and Cloud Computing.



Heming Cui is an assistant professor in computer science of HKU. His research interests include operating systems, programming languages, distributed systems, and cloud computing, with a particular focus on building software infrastructures and tools to improve reliability and security of real-world software. Homepage: https://i.cs.hku.hk/ heming/. He is a member of IEEE.